The Interim Experiment Report: A Systematic Account of The Experimental Design of Large Language Model-Based Text-to-Model Approaches

Sybren de Kinderen[1(✉)], Qin Ma[2], Jonathan Silva Mercado[2], and Karolin Winter[1]

[1] Department of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Eindhoven, The Netherlands
{s.d.kinderen,k.m.winter}@tue.nl
[2] University of Luxembourg, Esch-sur-Alzette, Luxembourg
{qin.ma,jonathan.silva}@uni.lu

Abstract. The increasing advent of approaches that adopt Large Language Models for text-to-model generation is accompanied by a variety of experimental designs for their evaluation. Inspired by "The Prompt Report", which provides a systematic account of prompting techniques for prompt engineering, we present a structured synthesis of experimental designs used to evaluate LLM-based text-to-model approaches.

Through a systematic literature review, we compile a compendium of experimental designs, organized into six tree-structured categories. This compendium integrates both experimental design principles from traditional empirical research and challenges unique to LLM usage. For example, specific to LLMs, we identify studies that conduct an ablation study to isolate the effect of individual prompting techniques, as well as those that address cascading errors introduced by intermediate LLM outputs. The compendium is intended to inform the design of future evaluations in this emerging field, offering researchers structured support and guidance.

Keywords: Experimental design · Text-to-model generation · Large Language Models

1 Introduction

With the increasing use of Large Language Models (LLMs) for conceptual (enterprise) modeling tasks [4,36], we also witness a growing body of approaches using LLMs to support automated translation of texts into a conceptual model. Examples of such LLM-based text-to-model approaches include domain modeling [2,7,12], process modeling [5,27], goal modeling [11] or otherwise.

The increasing advent of LLM-based text-to-model approaches is mirrored by a wide array of experimental designs for their evaluation. On the one hand, this diversity reflects general principles of empirical study design which apply regardless of whether an LLM is used, such as variation in datasets, participant populations, and methodological diversity (e.g., perceived quality assessed

Published by Springer Nature Switzerland AG 2026
H.-G. Fill et al. (Eds.): PoEM 2025, LNBIP 570, pp. 102–120, 2026.
https://doi.org/10.1007/978-3-032-12063-2_7

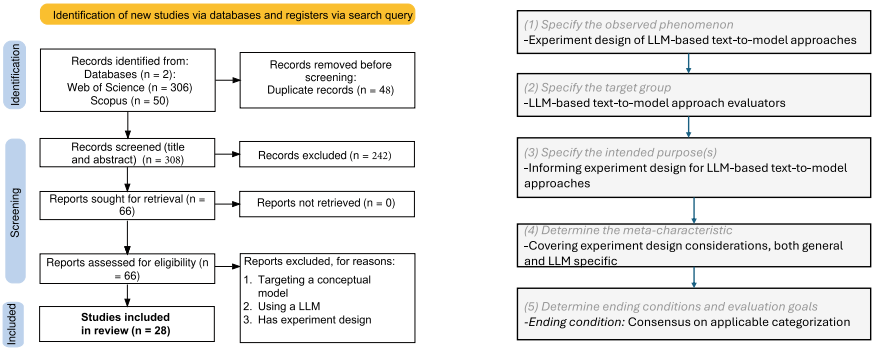
through qualitative surveys, as in [13,22,41] vs. ground truth-based quantitative performance metrics, as in [7,12,14,26,27,31]). On the other hand, and more intriguingly, we observe experimental designs specific to the use of LLMs and characteristics thereof. As highlighted by [9], who advocate standardized benchmarks of LLM-based (software) modeling, such considerations include (1) *solution space variability*, acknowledging that modeling tasks often allow for multiple valid solutions; (2) *non-determinism of LLM output*, where identical prompts can yield different results across runs; and (3) *prompting strategy dependency*, recognizing that the variations in prompt formulation, across a spectrum of prompting techniques, can significantly influence the generated LLM output.

Despite the recognition of both general and LLM-specific considerations in the experimental designs of LLM-based text-to-model approaches, a systematic account is still lacking. We argue that a structured synthesis of these experimental design practices, one that integrates both principles from traditional empirical research and challenges unique to LLM usage, holds significant potential. Such an account can serve as a valuable source of inspiration and guidance for experimental design of LLM-based text-to-model approaches, much like "The Prompt Report" [39] has done for prompt engineering. For example, as discussed in Sect. 3.7, [27] exemplifies a good practice by conducting an ablation study to isolate the effects of different prompting techniques on LLM output. Similarly, as discussed in Sect. 3.6 to address the variability of modeling solutions [5] gauges LLM-generated models against a reference model using multiple criteria: strict (exact syntax matching), relaxed (semantic equivalence), and flexible (allowing variation in level of detail). Accordingly, our research question reads *"What are the key considerations in the experimental design of Large Language Model-based text-to-model approaches?"*

To address this gap, we conduct a Systematic Literature Review (SLR) focusing on the experimental designs of studies that (1) employ an LLM, (2) target creating conceptual models from textual descriptions, and (3) provide a dedicated description of their experimental design. We contribute six overarching categories, each comprising of relevant sub-categories. These are:

- **Evaluation Aims**: the goals motivating the evaluation of LLM-based text-to-model based approaches;
- **Evaluation Focus**: what aspect is being evaluated, meaning, either the generated output or the dynamics of the modeling process;
- **Input and Output Dataset**: the nature of the input data and the resulting LLM output;
- **Evaluation Means**: how the approach is evaluated, such as comparison against a reference model or judgment based on specific properties;
- **Scoring Criteria**: the rubric used to grade LLM-generated output;
- **LLM-related Independent Variables**: the LLM specific variables that are systematically varied to assess their impact on LLM performance.

The remainder of this paper is structured as follows. In Sect. 2 we describe our SLR setup. In Sect. 3 we describe the experiment report, as a synthetic effort across the studies found in the SLR. Also we briefly describe the process for



(a) Filtering out the final paper selection according to PRISMA [29]

(b) Creating the categorization, according to [24]

Fig. 1. The SLR: reporting and analysis

deriving the categories. In Sect. 4 we conclude, with a wrap up of key learning, and the most important points going forward.

2 Literature Review Design

In order to provide a systematic account of the experimental design of LLM-based text-to-model approaches, we carry out a systematic literature review. Systematic literature reviews aim at synthesizing and structuring the current state of the art around a given topic, to identify gaps it [19]. For LLMs an example includes "The Prompt Report" [33], which surveys prompt engineering techniques and on which we base the title of this paper.

We report on our review following PRISMA, the Preferred Reporting Items for Systematic reviews and Meta-Analyses [29]. Figure 1a shows our process for arriving at the final paper selection.

Search Query. For our literature review we use the following search query:

("large language model" OR "generative AI") AND ("domain model" OR "conceptual model" OR "business process" OR "goal model" OR "enterprise model" OR "software model")

This keyword selection is primarily driven by our overarching research question (stated in the introduction). We follow our research question in terms of (1) the selection of conceptual enterprise models, of which we include various variants [19], pertaining to enterprise modeling and associated communities, like the business process management community. Also, we include the variants domain model and software model, which are especially prevalent in the Model Driven Engineering community; (2) the focus on LLMs, having generative AI as an associated variant. We have consciously excluded experiment-related keywords. This

is because the papers we target are inconsistent in their explicit characterization of the performed experiments in terms of their title, keywords, or abstract. Including experiment-related keywords, therefore, would unnecessarily narrow the selection of query results.

Identification. In the identification step, we queried Scopus and Web of Sciences, being two well established databases for scientific literature. While they exhibit a considerable overlap in terms of the retrieved papers (expressed in the n = 48 duplicates), nevertheless both have contributed to unique papers.

Screening. In the screening step, we first reviewed all retrieved unique papers on their relevance in terms of title and abstract. Here we primarily target the use of LLMs, and if the paper seems to address modeling, either directly (by virtue of targeting a modeling technique), or indirectly, by virtue of targeting a broader topic. Regarding the latter, one often mentioned broader topic is business process management, but since modeling is often a part of that, we did not want to exclude it from our initial set. We involved two authors in the screening on abstract and title of each paper, thereby ensuring that paper inclusion is consensus based. We excluded 242 out of 308 papers based on title and abstract, the main reason being that many papers were agreed upon to be thematically out of scope, often targeting a particular non-modeling application of LLMs.

Thereafter, we performed a full text reading of each remaining article. Next to a focus on modeling and LLMs being also important to the full text reading, an important inclusion criterion is the extent to which a paper has a dedicated discussion of its experimental design. We again relied on a consensus between two researchers from the pool of authors of this paper to make a decision.

Final Inclusion. After screening, 28 studies are included in our review.

3 The Interim Experiment Report

3.1 Creating the Categorization

Figure 1b displays an excerpt of the main steps for creating our categorization. Our design process takes cues from [24], though by no means we want to claim an exhaustive implementation of the associated taxonomy design process.

We focus on the experimental design for evaluating LLM-based text-to-model approaches (as a phenomenon, cf. [24]), with the goal of supporting approach evaluators (the target group) and informing their experimental design (the main purpose). To provide a comprehensive guidance, we address general and LLM specific experimental design considerations, and formulate the following meta-characteristic (cf. [24]): Covering both general and LLM specific experimental design considerations for evaluating LLM-based text-to-model approaches.

We define our categories complementarily. Firstly, we follow a conceptual-to-empirical approach, starting from predefined categories driven by (1) existing research like [9] for the category solution space variability, and (2) our research question, like for the category input dataset (as per the input text), output dataset

(as per LLM output), evaluation means and scoring criteria (for models, partially as tied to LLMs), and independent variables (which are particular to LLMs). We then adopt an empirical-to-conceptual approach, adding categories as they emerge from the reviewed literature and clustering them accordingly. For example, we added the category cascading error due to an experimental design in a reviewed study [40] that explicitly tackles this issue in a multi-step approach. As an ending condition (see Fig. 1b), we define reaching a consensus on a categorization we deem applicable.

Next, we discuss the resulting six categories.

3.2 Evaluation Aims

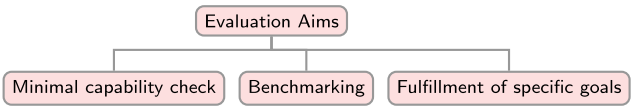


Fig. 2. Evaluation Aims category

The Evaluation Aims category (Fig. 2) captures the goals that motivate the evaluation of the reviewed LLM-based text-to-model approaches.

Minimal Capability Check. This aim investigates whether it is reasonable and practical to employ LLMs for conceptual modeling tasks. A substantial number of studies addressed this aim (e.g., [17,27,34,37]), but have articulated their evaluation goals using a variety of closely related formulations. Some focus on foundational questions, such as "whether it makes sense" to use LLMs for conceptual modeling [21] or to "explore the suitability of LLMs" [15]. Others aim to "check the ability" of LLMs [7] or to "understand the current capabilities of LLMs" in performing modeling tasks [41]. A further group of studies seeks to "assess the effectiveness of LLMs" [30], or to "investigate the extent to which LLMs can" construct accurate conceptual models [11]. Together, these formulations reflect a shared interest in evaluating the potential and readiness of LLMs for supporting conceptual modeling.

Benchmarking. This aim focuses on comparing LLM-based text-to-model approaches with alternative ones, including other LLM-based approaches [13,23], as well as non-LLM approaches such as rules-based systems [7], machine learning-based techniques [7], and human expert level performance [7,20].

Fulfillment of Specific Goals. This subcategory subsumes a diverse range of specific goals pursued in the evaluation of LLM-based text-to-model approaches. For example, in line with the Design Science Research (DSR) paradigm [18,41] takes stakeholder utility, defined as the extent to which stakeholder driven requirements are satisfied, as the primary and concrete evaluation criteria, in their

evaluation of a proprietary LLM-based solution for business process modeling within an organization, using its employees as evaluators.

In addition to user-defined requirements, specific purposes are also articulated through research questions or hypotheses. Examples include evaluating multi-perspective modeling and ensuring consistency [21]; evaluating compatibility of generated models with tools that require both XMI and diagrammatic representations [21]; and evaluating prompt effectiveness [5,10,11,25,40]. Other goals involve identifying different types of modeling elements [10,25], generating reusable reference models [30], and producing diverse and scalable synthetic training data [6,14,32]. Further studies examine the ability of LLMs to identify modeling patterns [40], their sensitivity to domains and modeling notations [10,11], the syntactic and semantic quality of generated models [10], the model size limitations [10], and robustness to intermediate result quality [5].

3.3 Evaluation Focus

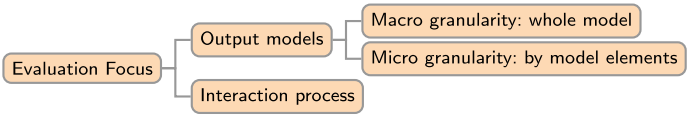


Fig. 3. Evaluation Focus category

The Evaluation Focus category (Fig. 3) classifies the evaluation targets set in experiments on the reviewed LLM-based text-to-model approaches. Specifically, it captures what is being evaluated. We observed a clear distinction between a majority of evaluations (e.g., [1,5,7,11,13,15,20,21,23,25,30,32,40]) focusing on generated output, namely the resulting models, and a relatively smaller number of evaluations (e.g., [10,41]) focusing on the interaction process, e.g., the dynamics of the modeling process itself.

Interaction Process [41] exemplifies an evaluation that puts the interaction, i.e. the modeling process, at the center, assessing user-perceived helpfulness. Another example is [10], which analyzes the complexity of interaction logs and conducts a strength and weakness analysis based on that to identify where LLMs perform well or struggle, and what common mistakes they tend to make.

Output Models. Output-oriented evaluations can be conducted at two levels of granularity: macro and micro. With the macro granularity, the generated models are assessed as a whole. The criteria used are often related to the intrinsic properties of the models, such as syntactic, semantic, or pragmatic validity [30]; understandability, correctness, completeness, relevance, and ease of updating [13]; alignment with the provided textual description [20]; or soundness and executability [23].

With the micro granularity, evaluations focus on individual model elements, often aggregated along the component-relationship dichotomy. For examples in goal modeling [11], components include intentional elements and actors, while relationships include contribution, decomposition, and dependency links. In multi-perspective enterprise modeling [21], components represent entities across different enterprise perspectives such as business, resource and technology, while relationships include both inter- and intra-perspective ones. In business process modeling [1,5,15,32], components include tasks, actors, and gateways, and relationships include flow and actor performer relation. For class diagram modeling [7,25,40], components consist of classes and attributes, while relationships encompass both associations and generalizations.

3.4 Input and Output Dataset

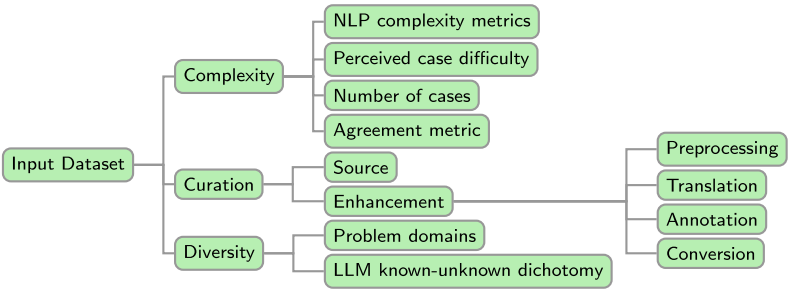


Fig. 4. Input Dataset category

The Input Dataset category (Fig. 4) concerns what LLM-based text-to-model approaches take as input during the experiments, which are mainly textual problem domain descriptions. When preparing input datasets for experiments, we observe efforts along three key subcategories: complexity, curation, and diversity, guided largely by empirical study principles.

Complexity. This subcategory concerns the coverage of different levels of complexity in input datasets. Several studies (e.g., [3,5,6,20,35]) target linguistic complexity using NLP complexity metrics such as the number of sentences, sentence length, number of words, number of unique words, and word length.

Notably, [3] goes beyond linguistic complexity to also consider perceived case difficulty. Specifically, both the readability of a case description measured by the Flesch-Kincaid Ease Score based on the number of sentences and word length and the estimated difficulty of cases as judged by the authors are used as indicators of input data complexity.

Additionally, many studies pay attention to the number of cases [12,13,26,27,34,40], often of different domains (cf. subcategory diversity below). Of note

here is [6] where creating a dataset is part of the paper contribution. They consciously compare their scenarios in terms of number of cases, in addition to ensuring coverage of different domains and levels of NLP complexity.

Finally, [7] uses an agreement metric (Fleiss' Kappa) as a proxy for input data complexity. The authors individually tag elements in an input scenario description to build a gold standard, and the level of agreement among them reflects the complexity of the scenario: the greater the tagging divergence, the higher the complexity.

Curation. This subcategory reflects the efforts made to achieve quality and fit-for-purpose input dataset from various sources. In terms of source, some studies rely on datasets from a curated repository, like the PET dataset for business process modeling [13,22,27], or a use case dataset for goal modeling and use case diagrams [2,7]. Alternative dataset sources include established scenarios from the literature, like in [15,34], from course material [6,7] or textbooks [21], and those created by the authors themselves [31]. Furthermore, [32] relies on LLMs to generate large synthetic datasets (covering 500 random domains).

A few studies [14,21,28,31] source and subsequently curate fit-for-purpose datasets through enhancement. Noteworthy is [14], who enhances a comprehensive Danish healthcare dataset via pre-processing (e.g., to remove duplications), translation (into English), and annotation (to make a PET-like dataset). Of interest, the authors use LLMs to support the annotation process to clarify ambiguities and improve annotating efficiency. When the source dataset lacks sufficient complexity variety (cf. subcategory complexity above), [21] enriches the dataset (which is considered complex) with a conversion of the original one into a simpler counterpart. Additionally, [28] applies three distinct strategies: base, flipped, and random, to convert non-textual source datasets into natural language representations with varying amounts of information incorporated. Finally, [31] converts simple descriptions in three distinct styles: analytical, which is precise and rigorous; eclectic, which is less structured but multi-perspective; and educational, starting with essential concepts only and adding details progressively.

Diversity. This subcategory concerns the range of domains represented in the input datasets. Many studies (e.g., [6,10,22,27,28,34,40]) explicitly report the problem domains covered in their experiments, spanning finance, energy, healthcare, education, transportation, or farming, to name a few. Particularly noteworthy for LLM-based text-to-model approaches, some papers, e.g. [7,10,11,28,37], deliberately accommodate the LLM known-unknown dichotomy in selecting the domains, including both well-represented and underrepresented domains in LLM training data in their input datasets.

In contrast, the Output Dataset category (Fig. 5) focuses on what LLM-based text-to-model approaches produce in the experiments, which of course primarily pertains to conceptual models. However, given the flexibility of LLM-generated text, these models can be represented variously, in terms of covered conceptual scope, used modeling notation, and specific output format.

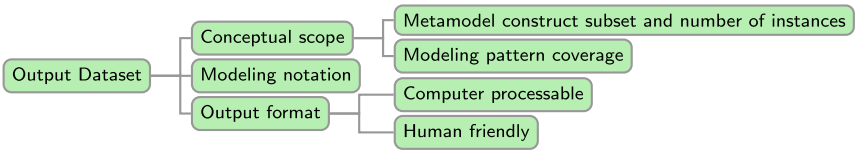


Fig. 5. Output Dataset category

Conceptual Scope. Many studies in their experiments consciously target only a metamodel construct subset, like [7] who, for UML class diagrams, covers only classes and associations, [31] who covers classes, attributes, and associations, whereas for process modeling with BPMN [22,26] cover a subset of process modeling concepts only. There are a few studies that aim at full conceptual coverage, but we find that this tends to be the case for modeling languages of a relatively small scale, like Data Flow Diagrams [17], or goal modeling languages [35].

Interestingly, a few studies explicitly target modeling patterns coverage in the output. Specifically, for UML class diagrams, [12] evaluates the output with respect to player-role and abstraction-occurrence patterns, finding that these patterns are rarely captured by the LLM. [40] follows up with a multi-step iterative automated domain modeling approach using LLMs, which according their evaluation improves the identification of player-role patterns.

Modeling Notation. While the conceptual scope subcategory above addresses variation in the use of the abstract syntax of modeling languages, [10] explores an equally noteworthy dimension in the design of the output dataset in experiments: the choice of modeling notations (i.e., concrete syntax). Specifically, [10] experiments how different UML notations: PlantUML and USE (the UML-based Specification Environment) affect LLM performance, with the finding that ChatGPT tends to make fewer syntactic mistakes when generating models in PlantUML.

Output Format. Experiments also vary in the expected output format. On the one hand, we find computer processable formats. This can be outputs directly importable into a modeling tool such as JUCMNav [34] for goal modeling, or PlantUML [3,31] for domain modeling. Others e.g., [12,22] take an indirect stance by requiring LLMs to generate structured formats, which then can be post-processed into conceptual models.

On the other hand, we observe studies e.g., [7,27] target human friendly formats in their experiments, without the aim of further computer processing. Often the output of these experiments is simply interpreted by the authors to gauge the performance against a reference model (cf. evaluation means, Sect. 3.5).

3.5 Evaluation Means

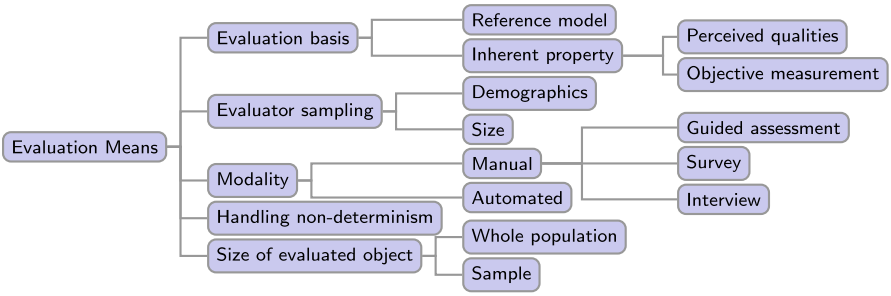


Fig. 6. Evaluation Means category

The Evaluation Means category (Fig. 6) describes how the LLM-based text-to-model approaches are evaluated (whereas the evaluation focus category presented in Sect. 3.3 describes what is being evaluated). We identify five subcategories based on the reviewed studies, detailed in the following.

Evaluation Basis. We observe that a majority of surveyed studies, e.g., [2,5–7,11,12,14,15,21,25–27,31,37,40] compare LLM generated models against a reference model. These studies typically employ information retrieval metrics (such as precision, recall, and F1 score) to gauge the accuracy of generated models, element by element. In addition, model distance metrics quantifying the difference between generated models and reference models to assess models as a whole, are sometimes employed, like in [5,16,26,28,37].

Alternatively, judgments are made based on the inherent properties of what is being evaluated, being either the generated models or the modeling process (cf. Section 3.3). These properties can be assessed subjectively, through perception of human evaluators (e.g., prospective users of the proposed approach [13,22,41] or study authors [3], cf. evaluator sampling subcategory discussed next), or objectively, using objective measurements, elaborated below.

In terms of perceived qualities, for example [13] checks understandability, correctness, completeness, and relevance of the generated business process models, as well as the ease of updating them, a property concerning the modeling process itself. [17] considers perceived completeness and correctness of generated models. And [41] assesses users' perception of the helpfulness, time-saving potential and facilitation of modeling tasks offered by their approach.

In terms of objective measurements, two studies stand out: [10] quantifies interaction complexity by tracking the number of sessions and prompts per session required for successful model generation; [32] measures size, density, separability and variability of generated models to judge how well they fit for constituting diverse and scalable synthetic datasets of declarative process models.

Evaluator Sampling. This subcategory touches on sampling considerations. The studies where LLM output is compared against a reference model often use the study authors as evaluators, who are well versed in the approach being assessed and evaluation criteria involved, such as [7,11,12,14,25,27,40].

In contrast, studies evaluating perceived qualities often recruit participants other than the study authors. Among the reviewed studies, both demographics and sample size are considered, but descriptions of evaluator demographics vary. [17] mentions three "evaluators" with no further details, whereas [13] mention "nine practitioners and professional developers from Bonitasoft and three academics specializing in BPM", without further specification. Others are more explicit in the participant demographics, with modeling experience [20,22] or BPM experience [41] being an explicitly discussed property of the sample. Furthermore, in case a survey focuses on user perceived qualities the population size is often mentioned, but only [22] remark on non response.

Modality. When it comes to the mode of evaluation, we find that studies perform their evaluation manually or automatically. Manual evaluation is used by the majority of studies. It is applied both in studies that compare the LLM output to a reference solution and studies in which perceived qualities are in focus (cf. evaluation basis subcategory above). When comparing to a reference model, studies employ guided assessment, following a grading rubric (cf. category scoring criteria discussed in Sect. 3.6 for more details), or established guidelines in the literature. For example, [32] applied two criteria in the Seven Process Model Guidelines (7PMG) to evaluate generated models. Another example is [7], who employ a decision tree for evaluators to qualitatively agree upon the status of a model element. Briefly, the tree evaluates element scope first, if in scope, evaluates redundancy, and if not redundant, considers relevance. When assessing perceived qualities, studies either use a survey as in [13,17,22], or conduct an interview with prospect users such as in [41] to collect feedback.

Only one study [28] conducts an automated evaluation of models (expressed in PDDL, Planning Domain Definition Language, a formal language to represent planning problems). The formal semantics of planning domain models and a heuristic model equivalence metric based on the semantics are essential components that make the evaluation computer amenable. However, as noted by [12], "automated comparison of two domain models is ... non-trivial ..., they either provide an imprecise estimation or work under carefully selected situations."

Handling Non-determinism. This subcategory, being specific to LLMs, considers the extent to which a study explicitly addresses the non-deterministic nature of LLMs, even when all the independent variables are fixed (cf. Section 3.7). Some studies, such as [7,15,23,40] systematically run the experiments multiple times and aggregate the results to account for the non-deterministic nature of LLMs. In contrast, one study [21] in our sample adopts a more pragmatic approach by re-running the experiments until the LLM generates a model importable to the targeted modeling tool, a prerequisite for further evaluation.

Size of Evaluated Object. Evaluation of generated models is often performed on the whole population. For example, studies evaluate all models generated from 7 [20], 15 [1] or 39 examples [5] within the PET dataset. Similarly, evaluations of UML models have covered all 8 [12,40] or 9 [7] exercises derived from university courses. However, when the number of models generated is too extensive to realistically evaluate, only a selected sample is evaluated. For example, in [32], 8 models were sampled for evaluation from a total of 500.

3.6 Scoring Criteria

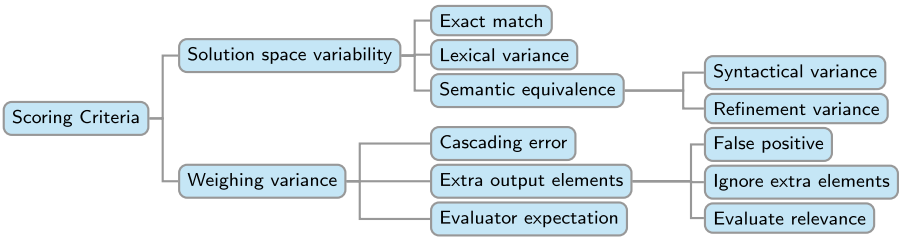


Fig. 7. Scoring Criteria category

The Scoring Criteria category (Fig. 7) encompasses the considerations involved in designing the scoring rubric used to grade LLM generated output.

Solution Space Variability. Conceptual modeling exhibits what [9] terms solution space variability, meaning that for same problem description multiple models can be equally valid. To account for this variability, the reviewed studies adopt a spectrum of scoring strictness levels. These different levels allow for the evaluation of LLM generated models within a broad solution space. As such, this accommodates diverse yet correct models.

Studies e.g., [5,25] use an exact match to check if the output model is identical to the reference. [5] refers to this as a strict comparison, allowing no deviation from the reference solution. [5,11,15,40] goes a step further by permitting lexical variance, namely allowing for synonymous names between the compared model elements. Even more accommodating are studies targeting semantic equivalence instead, realized in two forms. First, syntactical variance allows different model constructs to capture the same meaning [12,40]. For example, the reference model uses a boolean attribute to represent a status, while the LLM solution suggests an enumeration with two literals [12]. Second, refinement variance tolerates model constructs differing in the level of detail but are appropriate in the context. For example, [15] allows combining consecutive activities into a single one, while [5] allows splitting activities containing double action into two.

Weighing Variance. This subcategory addresses various strategies to weigh components (errors in LLM outputs or user perceptions) of evaluation.

When weighing errors, two types of errors are considered: cascading errors and extra output elements. Cascading errors originate from intermediate errors made in early stages of multi-step LLM-based text-to-model workflows. Scoring strategies may either weigh in intermediate errors by accounting for their downstream impact, such as in [40]. Alternatively, some evaluations weigh out intermediate errors by completely ignoring the cascading errors in the final result, as seen in [7]. Interestingly, one study [5] explores both settings: (1) an incremental setup, where early step results are used as input to later ones, allowing cascading errors to occur naturally, and (2) a gold standard setup, where each step is evaluated in isolation using the gold standard as input to prevent cascading errors.

Extra output elements are those generated by the LLM but not present in the reference model, (whereby the subcategory solution space variability discussed above concerns elements that in the reference model but not in the LLM output). A strict strategy for handling extra output elements is to treat them as false positives, such as in [11,40]. Alternatively, a more lenient strategy is to ignore the extra elements in scoring, such as in [7]. In between, there is also the strategy to evaluate the relevance of the extra elements and mark them as true or false positives accordingly, such as in [21]. In addition, [11,21] recommend reporting extra elements explicitly, regardless of how they are scored.

When weighing user perceptions, some studies suggest explicitly considering evaluator expectation and adjusting the weight of their perceptions accordingly. For example, negative opinion from a user with higher expectation should be less weighted than from a user with low expectation [41]. Another factor, highlighted by [20] is the user's awareness of whether a model is created by an LLM or a human, as this awareness can influence their perception and evaluation.

3.7 LLM-Related Independent Variables

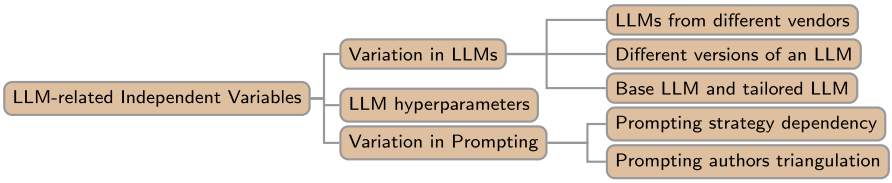


Fig. 8. LLM-related Independent Variables category

The LLM-Related Independent Variables category (Fig. 8) specifies subcategories that are systematically varied in LLM-based text-to-model experiments.

Variations in LLMs. Some studies evaluate the performance of LLMs from different vendors on the same modeling task. Comparisons include open-source models such as Llama and Starcoder [28], closed-source models like GPT and Gemini [23], and comparisons between open- and closed-source models, such as GPT

and Mixtral [7], or GPT, Cohere and Llama [34]. Other comparisons involve different versions of an LLM from the same organization. For closed-source, the model name is specified, such as GPT-4 and GPT-3.5-Turbo [16], or GPT-4o and GPT-3.5-0125 [27]. For open-source models, versions are typically categorized by LLM size, such as Llama 7B, 13B and 70B [28]. Finally, variations in LLMs can pertain to the use of both base models and tailored models. These include comparisons between the base model and the fine-tuned model for chat (e.g., Llama vs Llama-chat) [28], models fine-tuned with varying volumes of data [1], and general versus code specific LLMs for SQL generation [37].

LLM Hyperparameters. The behavior and output of LLMs can be configured through hyperparameters such as temperature and top-k. Briefly, the lower the values, the less variety in the generated responses. For example, [5,11,27,32,40] consciously set the temperature close to zero to avoid too variable outputs. [34] evaluates three different settings of temperature, 0.4, 0.6, and 0.8, and reports that the two higher values generated more acceptable goal models. Similarly, [28] adjusts the top-k parameter to manage the diversity of the generated responses.

Variation in Prompting The prompting strategy dependency is emphasized in a recent study [9], which highlights the influence of prompt formulations on the quality of models generated by LLMs. Our observation is that the most common approach is to modify the prompts then assess the impact on the generated models. These techniques include in-context learning approaches such as zero-shot, few-shot [8], and chain-of-thought [38], in which LLMs recognize and execute the task with the context provided in the prompt [8]. We found three studies comparing all three prompting techniques [12,25,27]. Two other studies compare zero-shot and few-shot [5,15]. Others use only one technique, either zero-shot [11,20] or few-shot [16,21,23,28,32,41]. Moreover, when an approach adopts prompting techniques decomposing the modeling task into consecutive steps, comparison with a single-step setting can be observed. For example, [40] divided into four steps, increasing the recall performance. [7] uses a two step decomposition achieving more accurate UML class models. [13] achieves higher accuracy in process models using six steps. Finally, [34] uses eight steps for goal modeling; however, it is not compared with other approaches.

We identified only one study that aimed to isolate the impact of individual prompting techniques through an ablation study. In [27], the authors set the LLM temperature to zero to reduce the model non-determinism so as to establish a baseline for comparison. Then they systematically removed specific prompting techniques from the baseline and reported quantitative results to assess the effect of each change in the prompt. However, other studies with prompt variation resemble an ablation study, even though they do not formally present it as such. These studies follow an approach to modify the prompt and compare the LLM performance. For example, [5,25,28] change the number of examples in few-shot prompting, or [11,20] use different instructions in zero-shot.

Another variation is prompting authors triangulation, where different authors independently create prompts for the same task. This approach evaluates the robustness of LLM outputs in response to variations in prompt formulation [15].

4 Concluding Outlook

In this paper, we used a systemic literature review to provide a synthesis of experimental design considerations of LLM-based text-to-model approaches. We organize our findings along six categories, with subcategories for each. Reflecting upon each category, we find the following observations to be of particular interest.

Evaluation Aims: Align with Research Questions and Stakeholder Needs. When evaluation goes beyond basic reality checks or benchmarking, it is good practice to explicitly articulate the evaluation goals through well-defined research questions and/or hypotheses. Moreover, in stakeholder-oriented context, evaluation should be driven by stakeholder needs, following principles of Design Science Research to ensure relevance and utility.

Evaluation Focus: The Process Can be Just as Important as the Output. Most reviewed studies emphasize evaluating the LLM-generated models, the final tangible artifacts. However, the modeling process itself including user interaction, iteration, and experience, is equally important, as it directly shapes the quality of the output. While the decision to evaluate models or process depends on the specific research focus, we find it valuable to consider both outcome and process when appropriate.

Input and Output Datasets: Address LLM Exposure Alongside Empirical Design Principles. While established empirical study design principles such as ensuring dataset complexity, diversity and quality remain essential, an equally critical factor specific to evaluations of LLM-based approaches is the LLM's prior exposure to the data. Evaluations should intentionally include both "known" and "unknown" datasets to enhance credibility of evaluation results.

Evaluation Means: Mitigate LLM Non-determinism to Support Reproducibility of Experiments. The inherent non-deterministic behavior of LLMs is a challenge for consistent evaluation of the experiments. Some studies address this by running the same experiment multiple times and aggregating the results, while others rely on LLM hyperparameters to reduce its effect to some extent. Clearly specifying the strategy adopted is essential to ensure some degree of reproducibility.

Scoring Criteria: Recognize Valid Solutions Beyond the Reference Models and Make the Criteria Explicit. Evaluating LLM-generated models using a single reference solution is a common practice, which may overlook potential valid alternatives. Moreover, cascading errors, namely errors in later step due to mistakes in early steps may penalize the overall score unfairly. Since there is no standardized scoring rubric, it is essential to clearly define: (1) how alternative correct solutions can be accepted (e.g., through a relaxed grading strategy), (2) how error propagation is handled, and (3) how these choices influence the overall evaluation. Making these assumptions explicit is important for ensuring transparent evaluation.

LLM-Related Independent Variables: Adopt a Formal Ablation Study to Isolate Effects. Multiple studies vary prompts to assess their performance;

however, only one study [27] explicitly presents results as an ablation study. This highlights an opportunity to isolate the impact of individual prompt components, supporting more interpretable and insightful evaluations.

Our goal is to support future evaluation of LLM-based text-to-model approaches by offering a compendium of experimental design considerations, both generally relevant and LLM-specific. In pursuing this goal, this paper has one key limitation: as is inherent to an SLR, our contribution is limited to a synthesis of the reviewed sample. While the sample certainly yields valuable insights into current experimental practice (cf. the summarized observations above), our SLR, at least in the way it was conducted, does not adopt a forward-looking perspective grounded in broader empirical context. For example, we did not incorporate insights from user studies in the empirical software engineering literature. Notably, many of the reviewed studies adopt an "engineering like" stance to evaluation, focusing on benchmarking LLM output on an often predefined dataset, and against predefined qualities. User focused considerations from the broader empirical (software engineering) literature, like an explicit account of user defined requirements, or a focus on user interaction with an LLM, appear underrepresented, at least in our sample.

Future work will aim to develop a final experiment report that includes two additional categories not covered in this paper due to space constraints: "evaluation metrics" and "reproducibility". Furthermore, we plan to integrate general user study guidelines from empirical software engineering to both guide experimental design and provide a forward-looking perspective complementing the findings of this review.

References

1. Apaydin, K., Zisgen, Y.: Local large language models for business process modeling. In: Delgado, A., Slaats, T. (eds.) ICPM 2024. LNBIP, vol. 533, pp. 605–609. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-82225-4_44
2. Arulmohan, S., Meurs, M., Mosser, S.: Extracting domain models from textual requirements in the era of large language models. In: ACM/IEEE International Conference on Model Driven Engineering Languages and Systems. MODELS, pp. 580–587. IEEE (2023)
3. Bari, D.D., Garaccione, G., Coppola, R., Torchiano, M., Ardito, L.: Evaluating large language models in exercises of UML class diagram modeling. In: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. ESEM, pp. 393–399. ACM (2024)
4. Barn, B.S., Barat, S., Sandkuhl, K.: Adaptation of enterprise modeling methods for large language models. In: Almeida, J.P.A., Kaczmarek-Heß, M., Koschmider, A., Proper, H.A. (eds.) PoEM 2023. LNBIP, vol. 497, pp. 3–18. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-48583-1_1
5. Bellan, P., Dragoni, M., Ghidini, C.: Process knowledge extraction and knowledge graph construction through prompting: a quantitative analysis. In: Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, pp. 1634–1641. ACM (2024)

6. Bozyigit, F., et al.: Generating domain models from natural language text using NLP: a benchmark dataset and experimental comparison of tools. Softw. Syst. Model. **23**, 1493–1511 (2024)
7. Bragilovski, M., Van Can, A.T., Dalpiaz, F., Sturm, A.: Leveraging machines to derive domain models from user stories. In: Requirements Engineering (2025)
8. Brown, T.B., et al.: Language models are few-shot learners. In: NeurIPS (2020)
9. Cámara, J., Burgueño, L., Troya, J.: Towards standarized benchmarks of LLMs in software modeling tasks: a conceptual framework. Softw. Syst. Model. **23**(6), 1309–1318 (2024)
10. Cámara, J., Troya, J., Burgueño, L., Vallecillo, A.: On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. Softw. Syst. Model. **22**(3), 781–793 (2023)
11. Chen, B., et al.: On the use of GPT-4 for creating goal models: an exploratory study. In: 31st IEEE International Requirements Engineering Conference. RE, pp. 262–271. IEEE (2023)
12. Chen, K., Yang, Y., Chen, B., López, J.A.H., Mussbacher, G., Varró, D.: Automated domain modeling with large language models: a comparative study. In: 26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems. MODELS, pp. 162–172. IEEE (2023)
13. Eldin, A.N., Assy, N., Anesini, O., Dalmas, B., Gaaloul, W.: A decomposed hybrid approach to business process modeling with LLMs. In: Comuzzi, M., Grigori, D., Sellami, M., Zhou, Z. (eds.) CoopIS 2024. LNCS, vol. 15506, pp. 243–260. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-81375-7_14
14. Grathwol, D., van der Aa, H., López, H.A.: Automating pathway extraction from clinical guidelines: a conceptual model, datasets and initial experiments. In: Comuzzi, M., Grigori, D., Sellami, M., Zhou, Z. (eds.) CoopIS 2024. LNCS, vol. 15506, pp. 296–312. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-81375-7_17
15. Grohs, M., Abb, L., Elsayed, N., Rehse, J.: Large language models can accomplish business process management tasks. CoRR (2023)
16. Guan, L., Valmeekam, K., Sreedharan, S., Kambhampati, S.: Leveraging pretrained large language models to construct and utilize world models for model-based task planning. In: Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems (2023)
17. Herwanto, G.B.: Automating data flow diagram generation from user stories using large language models. In: Joint Proceedings of REFSQ-2024 Workshops, Doctoral Symposium, Posters and Tools Track, and Education and Training Track, vol. 3672 (2024)
18. Hevner, A.R., March, S.T., Park, J., Ram, S.: Design science in information systems research. MIS Q. **28**(1), 75–105 (2004)
19. Kitchenham, B., Budgen, D., Brereton, P.: Guidelines for performing systematic literature reviews in software engineering. Technical report EBSE-2007-01, EBSE Technical report, Keele University and Durham University (2007)
20. Klievtsova, N., Benzin, J., Mangler, J., Kampik, T., Rinderle-Ma, S.: Process modeler vs. chatbot: is generative AI taking over process modeling? In: Delgado, A., Slaats, T. (eds.) ICPM 2024. LNBIP, vol. 533, pp. 637–649. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-82225-4_47
21. Kolev, P., Pruss, H.H., Wilken, J.R., Sandkuhl, K.: Grass-root enterprise modelling: how large language models can help. In: Paja, E., Zdravkovic, J., Kavakli, E., Stirna, J. (eds.) PoEM 2024. LNBIP, vol. 538, pp. 123–139. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-77908-4_8

22. Köpke, J., Safan, A.: Efficient LLM-based conversational process modeling. In: Gdowska, K., Gómez-López, M.T., Rehse, J.R. (eds.) BPM 2024. LNBIP, vol. 534, pp. 259–270. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-78666-2_20
23. Kourani, H., Berti, A., Schuster, D., van der Aalst, W.M.P.: Process modeling with large language models. CoRR (2024)
24. Kundisch, D., et al.: An update for taxonomy designers: methodological guidance from information systems research. Bus. Inf. Syst. Eng. **64**(4), 421–439 (2022)
25. Li, Y., Keung, J., Ma, X., Chong, C.Y., Zhang, J., Liao, Y.: LLM-based class diagram derivation from user stories with chain-of-thought promptings. In: 48th IEEE Annual Computers, Software, and Applications Conference. COMPSAC, pp. 45–50. IEEE (2024)
26. Licardo, J.T., Tankovic, N., Etinger, D.: A method for extracting BPMN models from textual descriptions using natural language processing. In: CENTERIS, vol. 239, pp. 483–490. Elsevier (2023)
27. Neuberger, J., Ackermann, L., van der Aa, H., Jablonski, S.: A universal prompting strategy for extracting process model information from natural language text using large language models. In: Maass, W., Han, H., Yasar, H., Multari, N. (eds.) ER 2024. LNCS, vol. 15238, pp. 38–55. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-75872-0_3
28. Oswald, J.T., Srinivas, K., Kokel, H., Lee, J., Katz, M., Sohrabi, S.: Large language models as planning domain generators. In: Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling. ICAPS, pp. 423–431. AAAI Press (2024)
29. Page, M.J., et al.: The Prisma 2020 statement: an updated guideline for reporting systematic reviews. BMJ (2021)
30. Piller, C.: Meta-prompt engineering in ChatGPT-4 for AI-generated BPM reference models. In: Elstermann, M., Lederer, M. (eds.) S-BPM ONE 2024. CCIS, vol. 2206, pp. 315–331. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-72041-3_22
31. Prokop, D., Stenchlák, S., Skoda, P., Klímek, J., Necaský, M.: Enhancing domain modeling with pre-trained large language models: an automated assistant for domain modelers. In: Maass, W., Han, H., Yasar, H., Multari, N. (eds.) ER 2024. LNCS, vol. 15238, pp. 235–253. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-75872-0_13
32. Santos, W.D.S., Coutinho, J.R., Baião, F., Spyrides, G.M., Lopes, H.C.V.: Terpsichora: a tool to generate synthetic MP-declare process models. In: Delgado, A., Slaats, T. (eds.) ICPM 2024. LNBIP, vol. 533, pp. 624–636. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-82225-4_46
33. Schulhoff, S., eta l.: The prompt report: a systematic survey of prompting techniques. CoRR (2024)
34. Siddeshwar, V., Alwidian, S.A., Makrehchi, M.: A comparative study of large language models for goal model extraction. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. MODELS, pp. 253–263. ACM (2024)
35. Siddeshwar, V., Alwidian, S.A., Makrehchi, M.: Goal model extraction from user stories using large language models. In: Bertolino, A., Pascoal Faria, J., Lago, P., Semini, L. (eds.) QUATIC 2024. CCIS, vol. 2178, pp. 269–276. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70245-7_19
36. Snoeck, M., Pastor, O.: Teaching conceptual modelling in the age of LLMs: shifting from model creation to model evaluation skills. Softw. Syst. Model. 1–11 (2025)

37. Song, Y., et al.: Enhancing text-to-SQL translation for financial system design. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, pp. 252–262. ACM (2024)
38. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: NeurIPS (2022)
39. White, J., et al.: A prompt pattern catalog to enhance prompt engineering with ChatGPT. CoRR (2023)
40. Yang, Y., Chen, B., Chen, K., Mussbacher, G., Varró, D.: Multi-step iterative automated domain modeling with large language models. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. MODELS, pp. 587–595. ACM (2024)
41. Ziche, C., Apruzzese, G.: LLM4PM: a case study on using large language models for process modeling in enterprise organizations. CoRR (2024)